Current Target Selector Implementation, Failure Propagation, and Active Hyperparameters

Internal Technical Note

March 26, 2026

Abstract

This note documents the *current* target-selector path used by the Lyapunov safety-filter implementation in the repository. The goal is not to describe the intended method in the abstract, but to state precisely what the code is solving now, what hyperparameters are active in the current notebooks, and how selector output is consumed by the safety filter. The current stack still requires an equilibrium package (x_s, u_s, d_s, y_s) to evaluate the Lyapunov certificate, but it no longer discards the last valid target outright when the current selector solve fails. Instead, the safety layer can reuse the last valid target as an operational backup target under the `target_backup_policy = last_valid` setting.

1 Direct Answer to the Current Diagnosis

The short answer is **not always anymore**: frequent target-selector failure can still increase fallback-MPC use, but the current implementation first attempts to reuse the last valid target package before declaring the target unavailable.

At each control step, the safety filter first requests a steady-state target package from the refined selector. The returned package is

$$\mathcal{T}_k = (x_s(k), u_s(k), d_s(k), y_s(k)) \,. \tag{1}$$

The Lyapunov check and the QCQP repair are both built around this package. If the selector returns

$$\texttt{target_info["success"]} = \texttt{False}, \tag{2}$$

then the current-step Lyapunov certificate is unavailable. In the present code path, the safety filter therefore does *not* solve the QCQP against a current-step target. It goes straight to the fallback branch, whose default is a fresh offset-free MPC action.

So the causal chain is

$$\text{target-selector failure} \implies \text{no current equilibrium package} \implies \text{no current Lyapunov certificate} \implies \text{candidate t} \tag{3}$$

2 Model and Coordinate Convention

The selector operates on the augmented offset-free linear model

$$z_{k+1} = A_{\text{aug}} z_k + B_{\text{aug}} u_k, \tag{4}$$

$$y_k = C_{\text{aug}} z_k, \tag{5}$$

with

$$z_k = \begin{bmatrix} x_k \\ d_k \end{bmatrix}, \qquad x_k \in \mathbb{R}^{n_x}, \quad d_k \in \mathbb{R}^{n_y}, \quad u_k \in \mathbb{R}^{n_u}, \quad y_k \in \mathbb{R}^{n_y}. \tag{6}$$

The implementation assumes the augmented state is ordered as

$$z_k = [x_k; d_k], \tag{7}$$

and also assumes

$$\dim(d_k) = n_y. \tag{8}$$

This assumption is explicit in the code and is part of the selector debug metadata.

The physical subsystem matrices used internally by the selector are extracted as

$$A = A_{\text{aug}}(1{:}n_x, 1{:}n_x), \tag{9}$$
$$B_d = A_{\text{aug}}(1{:}n_x, n_x{+}1{:}n_x{+}n_y), \tag{10}$$
$$B = B_{\text{aug}}(1{:}n_x, :), \tag{11}$$
$$C = C_{\text{aug}}(:, 1{:}n_x), \tag{12}$$
$$C_d = C_{\text{aug}}(:, n_x{+}1{:}n_x{+}n_y). \tag{13}$$

The disturbance frozen into the current target problem is

$$d_s(k) = \hat{d}_k. \tag{14}$$

All of these quantities are expected to be in the *same controller coordinates*. In the active notebook paths, that means scaled deviation coordinates.

3 What the Selector Solves Right Now

3.1 Steady-State Relations

At each sample, the target selector solves for an equilibrium satisfying

$$(I - A)x_s - Bu_s - B_d\hat{d}_k = 0, \tag{15}$$

and

$$y_s = Cx_s + C_d\hat{d}_k. \tag{16}$$

If an output-selection matrix H is supplied, the controlled output is

$$y_c = Hy_s. \tag{17}$$

In the current notebook paths, H is not supplied, so the implementation uses

$$H = I, \qquad y_c = y_s. \tag{18}$$

3.2 Stage 1: Exact Target Problem

The exact stage solves

$$\min_{x_s,u_s} (u_s - u_{\text{nom}})^\top R_u (u_s - u_{\text{nom}}) + x_s^\top Q_x x_s + \Phi_y^{\text{soft}}(y_s) \tag{19}$$

$$\text{s.t.} \quad (I - A)x_s - Bu_s - B_d \hat{d}_k = 0, \tag{20}$$

$$u_\ell \le u_s \le u_u, \tag{21}$$

$$y_c = y_{\text{sp}}, \tag{22}$$

$$y_\ell \le y_s \le y_u \quad \text{if output bounds are enabled.} \tag{23}$$

Here

$$u_\ell = u_{\min} + u_{\text{tight}}, \tag{24}$$

$$u_u = u_{\max} - u_{\text{tight}}. \tag{25}$$

If output tightening is used, the implementation similarly forms tightened output bounds. In the current notebook paths, no output bounds or tightening vectors are passed, so these terms are inactive.

If soft output bounds are enabled and output bounds exist, the code introduces nonnegative slacks

$$s_y^{\text{low}} \ge 0, \qquad s_y^{\text{high}} \ge 0, \tag{26}$$

and adds

$$\Phi_y^{\text{soft}}(y_s) = (s_y^{\text{low}})^\top W_y^{\text{low}} s_y^{\text{low}} + (s_y^{\text{high}})^\top W_y^{\text{high}} s_y^{\text{high}}. \tag{27}$$

However, because the active notebook paths do not pass output bounds, this softening machinery is currently inactive even though the software flag is left on.

3.3 Stage 2: Closest-Admissible Fallback Problem

If the exact stage is not accepted, the selector solves the fallback stage

$$\begin{aligned} \min_{x_s,u_s} \; & (y_c - y_{\text{sp}})^\top T_y (y_c - y_{\text{sp}}) + (u_s - u_{\text{nom}})^\top R_u (u_s - u_{\text{nom}}) + x_s^\top Q_x x_s \\ & + (x_s - x_s^{\text{prev}})^\top Q_{\Delta x} (x_s - x_s^{\text{prev}}) + (u_s - u_s^{\text{prev}})^\top R_{\Delta u} (u_s - u_s^{\text{prev}}) + \Phi_y^{\text{soft}}(y_s) \end{aligned} \tag{28}$$

$$\text{s.t.} \quad (I - A)x_s - Bu_s - B_d \hat{d}_k = 0, \tag{29}$$

$$u_\ell \le u_s \le u_u, \tag{30}$$

$$y_\ell \le y_s \le y_u \quad \text{if enabled.} \tag{31}$$

This stage removes the hard equality $y_c = y_{\text{sp}}$ and instead penalizes target error. In the active path:

$$Q_{\Delta x} = 0 \quad \text{because } Q_{\Delta x} \text{ is not provided,} \tag{32}$$

$$R_{\Delta u} \ne 0 \quad \text{because } R_{\Delta u} \text{ is supplied from } R_{\text{move}}. \tag{33}$$

So right now the fallback stage smooths the target *through the input target* but not through the state target.

4 Acceptance Logic

The selector accepts a stage only if the solver status and residual checks pass.

4.1 Accepted CVXPY Statuses

The implementation only accepts

$$\{\texttt{optimal}, \texttt{optimal_inaccurate}\}. \tag{34}$$

All other statuses are rejected immediately.

4.2 Residual Tolerances

The dynamic residual and exact-stage target residual are checked against a status-dependent tolerance:

$$\tau_{\text{dyn}} = \begin{cases} 10^{-6}, & \text{if status} = \text{optimal}, \\ 10^{-5}, & \text{if status} = \text{optimal_inaccurate}. \end{cases} \tag{35}$$

The exact stage is rejected if any of the following exceed the active tolerance:

$$\left\| (I - A)x_s - Bu_s - B_d \hat{d}_k \right\|_\infty , \tag{36}$$

$$\| y_c - y_{\text{sp}} \|_\infty , \tag{37}$$

$$\text{input/output bound violation.} \tag{38}$$

The fallback stage is rejected if the dynamic residual or bound violation exceeds the tolerance. If the fallback stage is accepted, its solve stage is stored as

$$\texttt{solve_stage} = \texttt{"fallback"}. \tag{39}$$

5 Returned Target Package

When successful, the filter-facing package is

$$\begin{aligned} \texttt{target_info} = \{&\texttt{success}, \texttt{x_s}, \texttt{u_s}, \texttt{d_s}, \texttt{x_s_aug}, \texttt{y_s}, \texttt{yc_s}, \\ &\texttt{requested_y_sp}, \texttt{solve_stage}, \texttt{target_error}, \texttt{target_error_inf}, \texttt{target_slack_inf}, \\ &\texttt{dyn_residual_inf}, \texttt{bound_violation_inf}, \texttt{warm_start}, \texttt{selector_debug}\}. \end{aligned} \tag{40}$$

When unsuccessful, the same structure is returned with

$$x_s = u_s = y_s = \varnothing, \tag{41}$$

while the debug section still stores solver status, reject reason, and residual information. That is exactly the failure mode that later causes the safety-filter branch to lose the current-step Lyapunov reference.

6 Why Selector Failure Triggers MPC Fallback

The Lyapunov safety filter uses the selected equilibrium to define

$$e_x = \hat{x}_k - x_s(k), \tag{42}$$

$$e_u = u - u_s(k), \tag{43}$$

$$V(e_x) = e_x^\top P_x e_x. \tag{44}$$

Without any valid (x_s, u_s, d_s) package, the current-step certificate

$$V(e_x^+) \leq \rho V(e_x) + \varepsilon \tag{45}$$

cannot be formed consistently around the current target.

In the present implementation, the control logic therefore behaves as

$$\text{if } \texttt{target_success} = \texttt{False and no last valid target exists} \implies \text{skip QCQP and call fallback MPC.} \tag{46}$$

This is why a run with many selector failures can still show many fallback-MPC activations, but only after the safety layer has exhausted the operational backup-target policy.

The current debug path also computes the post-action Lyapunov value against the effective target actually used by the safety logic, so the Lyapunov plots remain aligned with the runtime controller path.

7 Current Active Hyperparameters

7.1 Current Selector Hyperparameters in Both Safety-Filter Notebooks

The active notebook settings are the same for the RL-upstream and MPC-upstream experiments for the target selector itself. Table 1 lists the actual current settings.

Table 1: Current active target-selector settings in the two safety-filter notebooks.

Parameter	Value	Meaning in the current implementation
`Ty_diag`	$10^8 Q_{\text{out}}$	Very large penalty on fallback-stage output target error.
`Ru_diag`	**1**	Penalizes target input magnitude relative to u_{nom}.
`u_nom`	0	Because `u_nom_tgt=None`, the selector uses the zero vector in scaled deviation coordinates.
`Qx_diag`	None	No explicit diagonal passed. The code therefore uses $Q_x = w_x I$.
`w_x`	10^{-6}	Very small state regularization in both exact and fallback stages.
`Qdx_diag`	None	No target-state move penalty is active.

Parameter	Value	Meaning in the current implementation
`Rdu_diag`	R_{move}	Input target move penalty; defaults to `MPC_obj.R_in`.
`H`	None	All outputs are controlled; effectively $H = I$.
`solver_pref`	None	Uses `DEFAULT_CVXPY_SOLVERS = (OSQP, CLARABEL, SCS)`.
`prev_target`	previous successful target	Warm-start target package only if the previous solve succeeded.
`u_tight`	None	No input tightening.
`y_tight`	None	No output tightening.
`y_min`, `y_max`	None	No explicit output bounds passed.
`soft_output_bounds`	True	Present in code, but inactive because no output bounds are passed.

7.2 Current Safety-Filter Hyperparameters Used Around That Selector

The current notebook settings for the safety filter are shown in Table 2. These are the values used *after* the selector returns a target package.

Table 2: Current active safety-filter settings in the two safety-filter notebooks.

Parameter	Value	Meaning in the current implementation
`rho_lyap`	0.98	One-step contraction factor.
`lyap_eps`	10^{-9}	Additive numerical tolerance in the bound $V_{k+1} \leq \rho V_k + \varepsilon_\ell$.
`lyap_tol`	10^{-10}	Post-check tolerance for acceptance tests.
`w_track`	1.0	Weight on one-step output tracking in the QCQP.
`w_move`	0.2	Weight on moving away from the previous input.
`w_ss`	0.1	Weight on staying near u_s.
`w_mpc` / `w_rl`	1.0	Weight on staying close to the upstream candidate.
`allow_lyap_slack`	Notebook-specific	Enables Lyapunov slack as a solver aid, but hard-only acceptance is still the default.
`trust_region_delta`	Notebook-specific	Optional trust region around the candidate action.
`allow_trust_region_slack`	False	Trust region stays hard unless explicitly softened.
`tracking_target_policy`	`raw_setpoint`	Upstream MPC, fallback MPC, and QCQP output term use one normalized tracking target policy.
`target_backup_policy`	`last_valid`	Reuse the last valid target package when the current selector solve fails.

Parameter	Value	Meaning in the current implementation
`selector_warm_start`	True	Reuse the previous target as a CVXPY warm start when available.
`Qdx_tgt_diag`	$10^{-6}\mathbf{1}$	Small state-target smoothing term on the fallback selector stage.
`lyap_acceptance_mode`	`hard_only`	Only hard-feasible corrected actions are accepted as verified control.
`reuse_mpc_solution_as_ic`	False	MPC solve starts from the fixed provided `IC_opt` every step.
`reset_system_on_entry`	True	The runner restores the plant object to its entry snapshot on each run.

8 Implications of the Current Settings

The active selector weights imply the following behavior.

1. The fallback stage strongly prefers output-setpoint matching because $T_y = 10^8 Q_{\text{out}}$ is very large.
2. The target state is only weakly regularized because $w_x = 10^{-6}$ is very small.
3. There is now a small explicit state-target smoothing term because $Q_{\Delta x}$ is active with a default diagonal of 10^{-6}.
4. There *is* input-target smoothing because $R_{\Delta u}$ is inherited from the move-weight vector.
5. The selector now uses a genuine solver warm start when a previous target is available.
6. Because there are no active output bounds in the current notebook path, the soft-output-bound machinery does not currently explain the failures.

So if the selector is failing often, the most relevant active ingredients to inspect are:

$$T_y, \quad R_u, \quad R_{\Delta u}, \quad \hat{d}_k, \quad y_{\text{sp}}, \quad \text{and solver status.} \tag{47}$$

9 New Debug Data Added for This Issue

The debug exporter now stores selector-status trajectory data for the entire run:

$$\texttt{target_success_flags}, \quad \texttt{target_failure_flags}, \quad \texttt{target_stage_code}. \tag{48}$$

The stage coding is

$$0 = \text{failed}, \tag{49}$$
$$1 = \text{fallback stage accepted}, \tag{50}$$
$$2 = \text{exact stage accepted}. \tag{51}$$

The exporter also now generates a full-trajectory plot

$$\texttt{target_selector_status.png}, \tag{52}$$

whose top subplot shows selector success/failure over the entire trajectory and whose bottom subplot shows the stage code over the entire trajectory.

10 Practical Interpretation

If a saved run shows:

1. many `target_failure_flags = 1`,
2. many `correction_mode = fallback_mpc_unverified` or `fallback_mpc_verified`,
3. few QCQP corrections,

then the main bottleneck is not the QCQP. The main bottleneck is target availability. In that situation, the correct debugging priority is:

1. diagnose why the selector fails,
2. only then retune the QCQP.

11 Next Concrete Checks

The next debugging pass should inspect, step by step:

1. `target_selector_status.png`,
2. `target_error_inf`, `target_slack_inf`, and `target_mismatch_inf`,
3. selector solver status and reject reasons inside `selector_debug`,
4. disturbance-estimate evolution $\hat{d}_k$ around the first failure burst,
5. whether the requested setpoint is locally incompatible with the current disturbance estimate.

Only after that should the QCQP weights be treated as the main suspect.